M2 EEMI · SOUTENANCE FINALE 2026

Repère

Réserver un espace de coworking à l'heure, et prouver qu'on y est.

Ilias Benharrat

Next.js /20 | React Native /20 | Docker /20

LE PRODUIT

Un endroit pour travailler, libre avant de se déplacer

Repère référence des lieux de coworking. Un membre réserve un espace à l'heure et paie en crédits : débités à la réservation, rendus s'il annule avant le début.

Visiteur

Découvre les lieux, les types d'espace et les tarifs, puis crée un compte.

Membre

Réserve, consulte et annule ses réservations, valide son arrivée sur place, gère son compte.

Administrateur

Gère les lieux, les espaces, les utilisateurs et les réservations.

UN PRODUIT, TROIS NOTES

Trois compétences sur la même base de données

Note Next.js

Le site web

Next.js 16.3, App Router. Site public, authentification, onboarding, espace membre, réservation, paramètres, back-office.

Note React Native

L'application mobile

Expo SDK 57, Expo Router. Réserver près de soi, scanner le QR code de l'espace, valider son arrivée par la position.

Note Docker

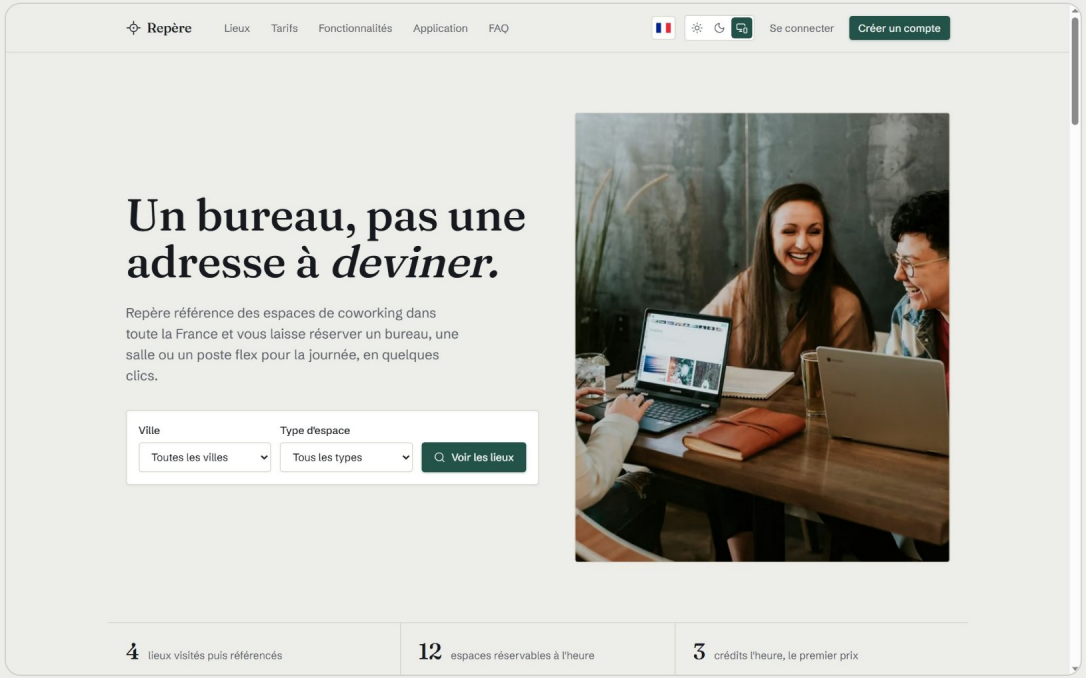
L'image du site

Image de production du site, Compose avec PostgreSQL et les migrations, scan Docker Scout.

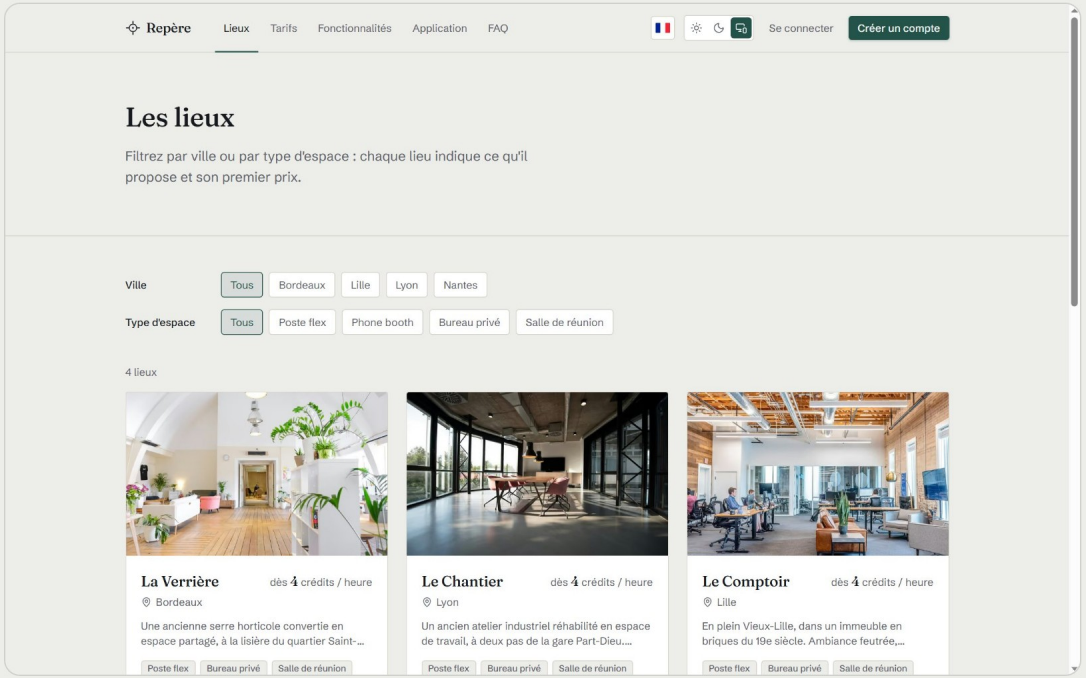
Une seule base PostgreSQL. Le site y accède par Prisma ; l'application passe par l'API /api/mobile/v1 du site. Une réservation faite d'un côté se voit de l'autre.

DÉMO NEXT.JS · SITE PUBLIC

Le produit existe avant la connexion



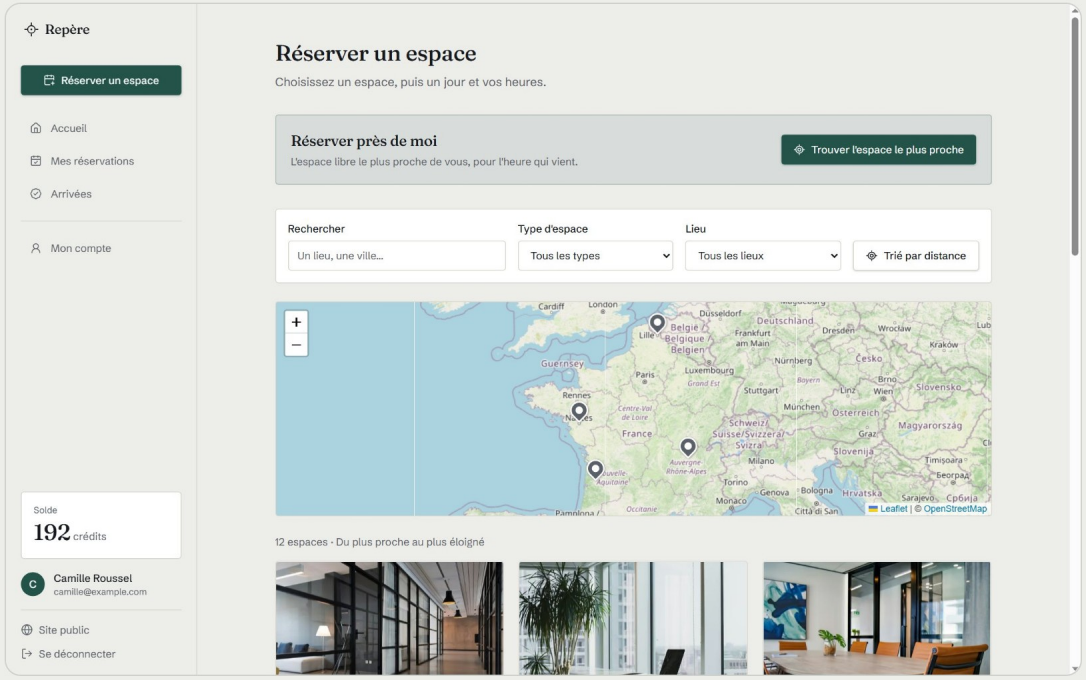
Accueil : recherche par ville et par type d'espace, chiffres et prix lus en base.



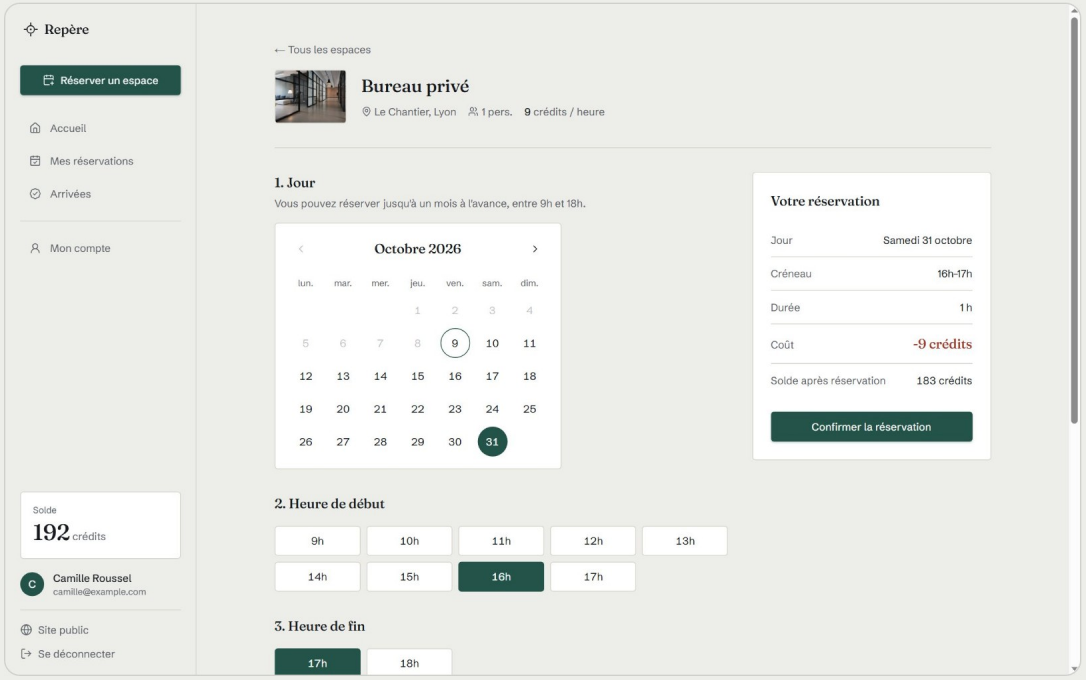
/lieux : filtre par searchParams, formulaire GET qui fonctionne sans JavaScript.

DÉMO NEXT.JS · ESPACE MEMBRE

Réserver : un parcours qui persiste en base



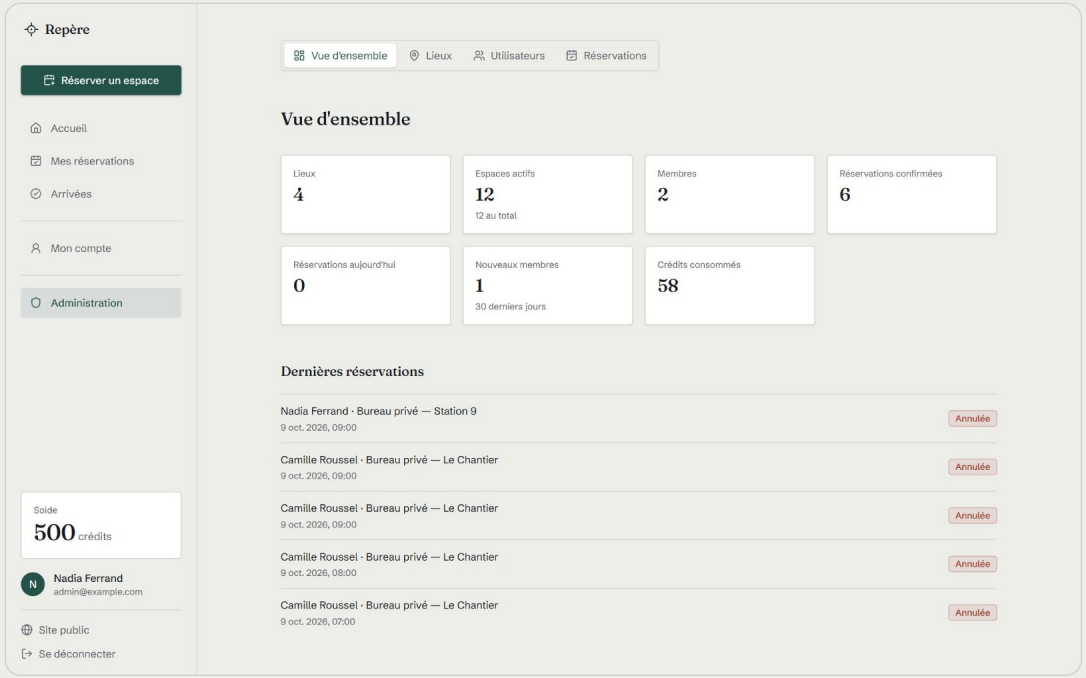
/reserver : recherche, carte des lieux, tri par distance, « Réserver près de moi ».



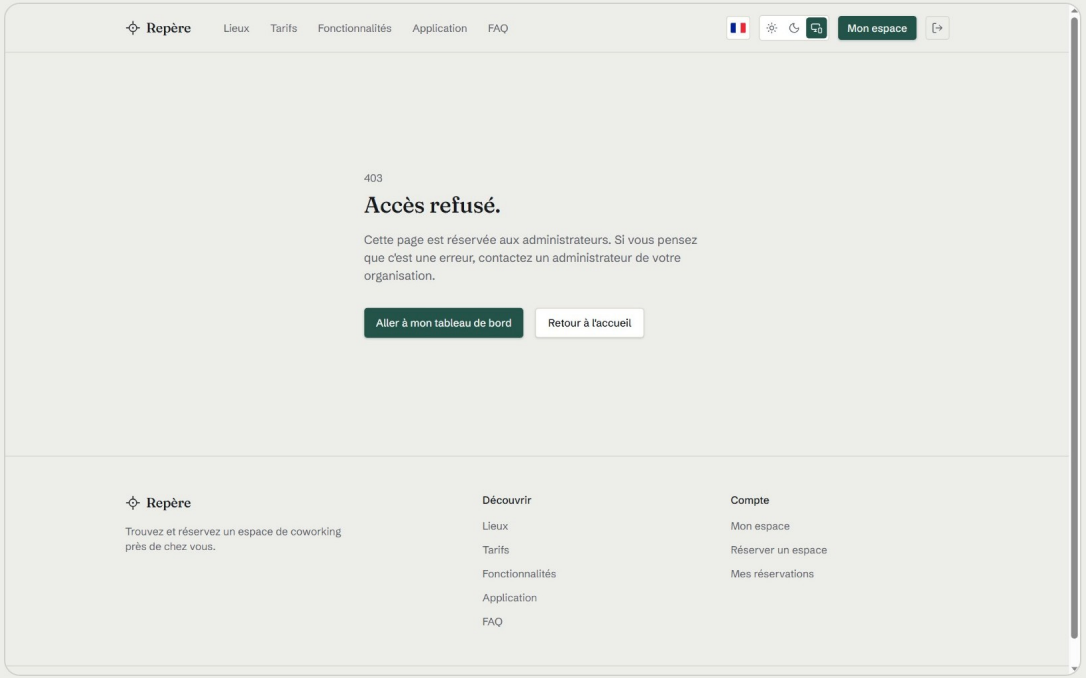
/reserver/[spaceId] : calendrier, heures, coût et solde restant avant de confirmer.

DÉMO NEXT.JS · BACK-OFFICE

Un rôle administrateur, protégé côté serveur



/admin avec le compte administrateur : statistiques, lieux et espaces, utilisateurs, réservations.



La même adresse avec le compte membre : accès refusé, décidé par le serveur.

ARCHITECTURE · APP ROUTER

Quatre groupes de routes, trois niveaux de layouts

Dossier de src/app	Responsabilité
(marketing)	Site public, indexable
(auth)	Connexion, inscription, déconnexion
(onboarding)	Profil obligatoire après l'inscription
(app)	Espace membre, et admin/ avec son propre layout
api/mobile/v1	Route Handlers JSON de l'application mobile

Serveur par défaut

Toutes les pages et tous les layouts sont des Server Components.

Client à la feuille

40 composants client, classés par raison dans la documentation.

Colocation

Composants et actions d'une route vivent à côté d'elle.

CHOIX DE RENDU · SERVEUR, CLIENT, CACHE

Trois choix de rendu, et pourquoi

Rendu serveur

/lieux filtré par l'URL

La page lit searchParams et filtre sur le serveur. Aucun état client : la vue se partage, et la page marche sans JavaScript.

Zone interactive client

Le sélecteur de créneau

Calendrier et heures en état local, pour répondre au clic. Il ne fait que proposer : la Server Action revérifie tout.

Donnée cachée ou dynamique

Lieux en cache, solde en direct

Les lieux publics sont mis en cache et invalidés par tag. L'espace membre lit sans cache : un solde doit être exact.

Vérifié : en modifiant les champs cachés du formulaire pour demander un créneau à 7 h, l'action a refusé.

BASE DE DONNÉES · POSTGRESQL 16 ET PRISMA 7

Huit tables, des écritures en transaction

Table	Contenu
users	Comptes, rôle, solde de crédits
locations	Lieux, avec coordonnées GPS
spaces	Espaces : type, prix par heure, statut
reservations	Un espace, un membre, un créneau
preferences	Préférences d'un membre
sessions	Sessions du site, par empreinte du jeton
mobile_sessions	Sessions de l'application mobile
check_ins	Tentatives d'arrivée, acceptées ou refusées

Relations

Un lieu contient des espaces. Un membre réserve un espace. Une réservation reçoit des tentatives d'arrivée.

Transactions

Réserver crée la réservation et débite les crédits ensemble, ou rien. Isolation SERIALIZABLE, rejouée jusqu'à 3 fois en cas de conflit.

Migrations

Quatre migrations versionnées. Les trois dernières n'ajoutent que des tables ou une colonne facultative.

Schéma : prisma/schema.prisma · détail : docs/base-de-donnees.md

Note Next.js · données réelles et persistantes

DONNÉES · LECTURES ET ÉCRITURES

Lire sur le serveur, écrire par Server Action

Page
Server Component : lit la base par Prisma

→

Formulaire
Composant client : saisie, état de l'envoi

→

Server Action
Garde d'accès, puis validation zod

→

Transaction
Créneau libre, débit des crédits

→

Invalidation
revalidatePath et revalidateTag

20 Server Actions, dans 13 fichiers _actions.ts

2 familles de Route Handlers : la déconnexion et l'API mobile

17 points d'entrée JSON pour l'application mobile

AUTHENTIFICATION, AUTORISATION, SÉCURITÉ

Le serveur décide, l'interface ne fait qu'afficher

Session

Un jeton opaque dans un cookie. Seule son empreinte SHA-256 est enregistrée en base.

Mots de passe

Hachés avec scrypt. Un changement de mot de passe ferme les autres sessions.

Gardes

requireUser, requireOnboarded, requireAdmin : dans les layouts et en première ligne de chaque Server Action.

Propriété

Chaque action revérifie que la réservation appartient à l'utilisateur connecté.

Frein de connexion : 10 échecs en 10 minutes par adresse, commun au site et à l'API. Aucune variable NEXT_PUBLIC_, aucune clé exposée.

PERFORMANCE, CACHE, SEO, ÉTATS, QUALITÉ

Ce qui se mesure

97–100

Lighthouse, performance, profil mobile

100

accessibilité, bonnes pratiques et SEO

80

tests unitaires des règles métier

24

fichiers loading.tsx, un squelette par page

Cache. Les lieux et les espaces publics sont mis en cache, puis invalidés par tag dès qu'un administrateur les modifie.

États. Chargement, erreur avec « Réessayer », page introuvable, liste vide, erreurs sous chaque champ, toasts.

REACT NATIVE · POURQUOI CETTE APPLICATION

Le téléphone sert au moment d'arriver

Il sait où je suis

Les espaces sont classés par distance, et l'application propose le plus proche qui est libre maintenant.

Il prouve l'arrivée

Le serveur n'accepte l'arrivée qu'à 150 m ou moins du lieu ; le QR code dit de quel espace il s'agit.

Il reste sous la main

La prochaine réservation et l'historique restent lisibles sans réseau.

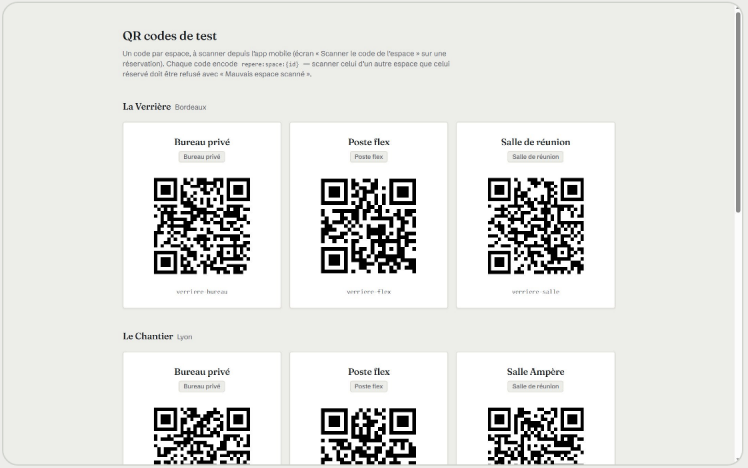
Démonstration sur iPhone, dans Expo Go

Note React Native · parcours et valeur d'usage 2 pts

REACT NATIVE · SCAN QR ET GÉOLOCALISATION

Le scan déclenche une action métier

Étape	Ce qui se passe
QR code de l'espace	Il contient repere:space: suivi de l'identifiant
Caméra	Permission demandée à l'ouverture du scan
Contrôle local	Préfixe vérifié avant tout appel réseau, un seul scan traité à la fois
Envoi	Identifiant scanné et position du téléphone
Serveur	Bon espace, heure, précision, distance
Retour	Arrivée acceptée ou refusée avec le motif, tracée dans l'historique



Acceptée à 150 m ou moins du lieu, de 15 minutes avant le début à la fin du créneau, avec une position précise à 100 m ou mieux.

QR codes de test : page /qrcode du site, compte administrateur

Note React Native · scan QR 2 pts · géolocalisation 2 pts

REACT NATIVE · ARCHITECTURE ET BACKEND

Un écran orchestre, il ne porte pas la logique

Dans l'application

1. Écran, dans src/app : Expo Router
2. Hook et règles, dans src/features
3. Service, dans src/services
4. apiFetch : le seul appel réseau

Jusqu'à la base

1. iPhone, Expo Go
2. Tunnel HTTPS
3. Passerelle : /api/mobile/v1 seulement
4. Next.js, puis PostgreSQL

Session et données

- Jeton dans le trousseau d'iOS
- Erreur 401 : déconnexion
- Cache conservé hors ligne
- 136 tests unitaires

Je développe sur un poste d'entreprise en bureau à distance, sans droits d'administrateur : ses ports ne sont pas joignables depuis le téléphone. D'où le tunnel, et une passerelle qui n'expose ni le site ni le back-office.

INFRASTRUCTURE · HÔTE, CONTENEURS, TÉLÉPHONE

Qui joint quoi, et par quelle adresse

Depuis	Le site	La base
Mon navigateur, sur l'hôte	localhost:3000	localhost:5433
Le conteneur web	C'est lui : il écoute sur 0.0.0.0:3000	postgres:5432, par le nom du service
Un docker run seul	localhost:3000	host.docker.internal:5433
Le téléphone	Adresse HTTPS du tunnel, API seulement	Jamais directement

Image
Le modèle figé, produit par docker build.

Conteneur
Une image en cours d'exécution, avec ses variables.

Port publié
3000 de l'hôte vers 3000 du conteneur ; 5433 vers 5432.

Volume
Les données de PostgreSQL survivent à l'arrêt.

Sur le téléphone, localhost désigne le téléphone : d'où le tunnel

DOCKER · LA DÉMARCHE

Du projet local à l'image reproductible

1. Vérifier le build

npm run build passe hors Docker ; Next.js est réglé en sortie standalone.

2. Réduire le contexte

.dockerignore écarte node_modules, .next, .git, .env et les docs.

3. Écrire le Dockerfile

Plusieurs étapes : construire avec tout, ne livrer que le nécessaire.

4. Sortir la base du build

Aucune page ne lit la base à la construction ; DATABASE_URL arrive au démarrage.

5. Tester dans le conteneur

Connexion, réservation, annulation, refus de /admin, sur une base vierge.

6. Scanner, corriger, rescanner

Docker Scout, corrections, reconstruction, nouveau scan.

Un bug que seul le test de l'image a montré : la déconnexion redirigeait vers 0.0.0.0. Corrigé par une redirection relative.

DOCKER · L'IMAGE DE PRODUCTION

Quatre étapes, une image de 256 Mo

Étape	Rôle
deps	npm ci sur les seuls fichiers package : couche mise en cache
builder	next build, sortie standalone
runner	node server.js, utilisateur node, healthcheck, sans npm
migrate	Migrations et données de démonstration, image à part

.dockerignore

Ni node_modules, ni .git, ni .env : contexte de build de 3,85 Mo.

Variables

DATABASE_URL est lue au démarrage du conteneur, jamais pendant le build.

Rien de secret dans l'image

Aucun fichier .env, aucune trace de DATABASE_URL dans l'historique des couches.

DOCKER · LANCEMENT ET VARIABLES

Une commande, trois services dans l'ordre

postgres
PostgreSQL 16. Un healthcheck dit quand il est prêt ; un volume garde les données.

→

migrate
Applique les migrations, charge la démo si la base est vide, puis s'arrête.

→

web
Démarre quand migrate a réussi. Déclaré « healthy » quand le site répond.

```
docker compose up --build
docker compose ps
docker compose logs -f web
docker compose down
```

Variables et secrets

Une seule variable, DATABASE_URL. Compose la donne au conteneur ; docker run la lit dans un fichier .env.docker. Aucun .env n'entre dans l'image : ils sont ignorés par Git et par .dockerignore.

docker compose down garde les données : elles sont dans le volume

Note Docker · lancement 2 pts · variables 2 pts

DOCKER · SCAN SCOUT ET CORRECTIONS

De 26 vulnérabilités critiques et hautes à zéro

Image	Critiques	Hautes	Moyennes	Paquets	Taille
1. node:24-bookworm-slim	5	21	19	364	304 Mo
2. node:24-alpine, sans npm	1	2	0	112	245 Mo
3. + next 16.3.8, sharp 0.35.5, apk upgrade	0	0	0	113	251 Mo

Lecture

24 des 26 venaient de l'image de base et de la CLI npm, pas du code du projet.

Refusé après test

generateStaticParams vide, URL de base factice au build, image distroless, npm audit fix --force.

Revérifié le 9 octobre

Image reconstruite sans cache depuis le dépôt : toujours zéro.

USAGE DE L'IA

L'agent écrit le code, je cadre, je décide, je valide

Outils

- ChatGPT : le prompt de départ
- Claude Code : plan, code, tests, documentation
- Claude Design : maquettes
- Jira par MCP : sprints et tickets

Cadre

- Des règles écrites
- Des skills tirés des docs officielles
- Un fichier de contexte tenu à jour
- Des questions avant de coder
- Aucun commit sans ma validation

Ce que j'ai refusé

- Web : generateStaticParams vide, qui donnait une erreur 500 dans le conteneur
- Mobile : un bouton « Confirmer quand même » sur une arrivée refusée

Ce que l'agent ne voit pas : le téléphone. Deux bugs n'ont été trouvés qu'en testant sur l'iPhone.

LIMITES CONNUES

Ce que le projet ne fait pas

Site et Docker

- Pas de déploiement en ligne
- Tests unitaires seulement, pas d'intégration continue
- Frein de connexion en mémoire du processus
- Aucun e-mail : ni réinitialisation, ni vérification
- Le back-office ne supprime pas un lieu

Application mobile

- iOS uniquement, dans Expo Go
- Pas de NFC, pas de development build
- La position vient du téléphone : elle peut être falsifiée
- Hors ligne : lecture seule
- Le backend tourne sur mon poste, atteint par un tunnel

QUESTION FINALE

Si vous clonez le dépôt, trois commandes suffisent

```
git clone https://github.com/Benbecker69/next_js_m_deux.git
cd next_js_m_deux
docker compose up --build
```

Ouvrir

http://localhost:3000

Comptes de démonstration

camille@example.com et admin@example.com, mot de passe demo1234

Se repérer dans le code

README en guide de correction, et la table « Où intervenir pour… » de docs/architecture.md